Reinforcement Learning: Tutorial 9

Deep Reinforcement Learning

Week 6
University of Amsterdam

Alejandro Garcia
March 2026

Outline

1. Deep Reinforcement Learning

2. Ask anything about HW4

Tutorial 9 Overview

1. Deep Reinforcement Learning
2. Ask anything about HW4

Tutorial 7 Overview

1. Deep Reinforcement Learning
   - Questions 8.1-8.2
2. Ask anything about HW4

Theory Intermezzo: DQN

1. **Initialization.** θ and ϕ, replay buffer B, initial state s_1
2. **Iterations:** For every $t \geq 1$,
   compute π_t the ϵ-greedy policy w.r.t. Q_θ
   take action a_t according to π_t, and observe r_t, s_{t+1}
   store (s_t, a_t, r_t, s_{t+1}) in B
   sample k experiences (s_i, a_i, r_i, s'_i) from B
   for $i \in [1, k]$: compute using the target net. with weights ϕ

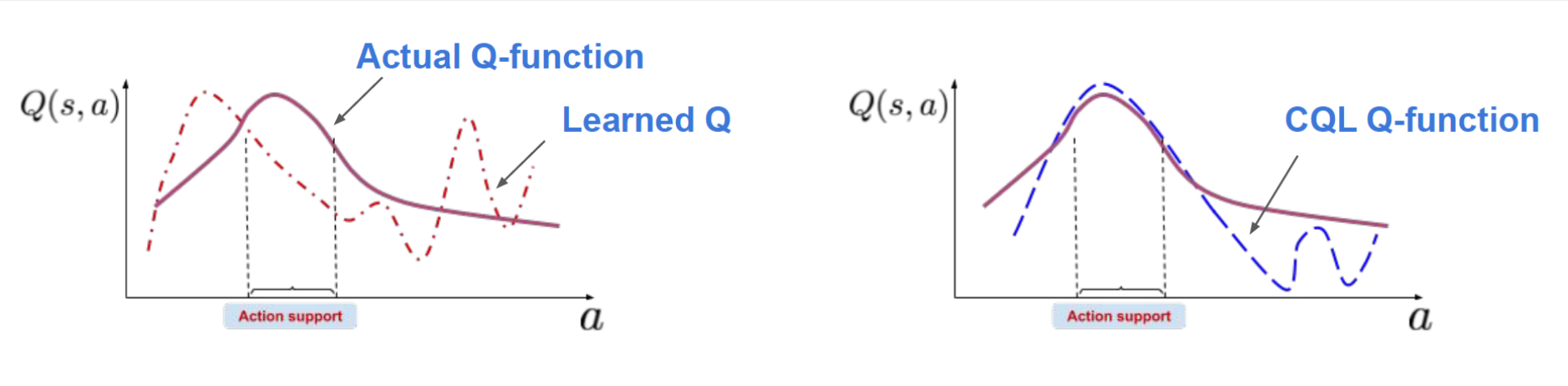
$$y_i = \begin{cases} r_i & \text{if episode stops in } s'_i \\ r_i + \lambda \max_b Q_\phi(s'_i, b) & \text{otherwise} \end{cases}$$

   update the weights θ (using back prop.) as:

$$\theta \leftarrow \theta + \alpha(y_i - Q_\theta(s_i, a_i))\nabla_\theta Q_\theta(s_i, a_i)$$

   every C steps: $\phi \leftarrow \theta$

Theory Intermezzo: CQL



Theory Intermezzo: CQL

minimize big Q-values

maximize data Q-values

$$\min_{Q} \max_{\mu} \; \alpha \left(\mathbb{E}_{\mathbf{s}\sim\mathcal{D},\mathbf{a}\sim\mu(\mathbf{a}|\mathbf{s})} \left[Q(\mathbf{s},\mathbf{a}) \right] - \mathbb{E}_{\mathbf{s}\sim\mathcal{D},\mathbf{a}\sim\hat{\pi}_{\beta}(\mathbf{a}|\mathbf{s})} \left[Q(\mathbf{s},\mathbf{a}) \right] \right)$$
$$+ \frac{1}{2} \mathbb{E}_{\mathbf{s},\mathbf{a},\mathbf{s}'\sim\mathcal{D}} \left[\left(Q(\mathbf{s},\mathbf{a}) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}^k(\mathbf{s},\mathbf{a}) \right)^2 \right] + \mathcal{R}(\mu) \quad (\mathrm{CQL}(\mathcal{R}))$$

standard Bellman error

Q 8.1 Optimal policies in CQL

In this exercise we will show the type of policy obtained by maximizing an entropy-regularized objective $\mathbb{E}_{a\sim\pi(a)}[Q(a)] + \mathcal{H}(\pi)$ as used in conservative Q learning. $\mathcal{H}$ denotes the entropy. We will look at a policy $\pi(a)$ for a one-state MDP, thus, states can be omitted from any notation.

1. Write the objective in the form $\sum_a \ldots$.

Q 8.1 Optimal policies in CQL

1. Write the objective in the form $\sum_a \ldots$.

$$\begin{aligned}\mathbb{E}_{a\sim\pi(a)}[Q(a)] + \mathcal{H}(\pi) &= \sum_a \pi(a)Q(a) - \sum_a \pi(a)\log(\pi(a)) \\ &= \sum_a \pi(a)(Q(a) - \log(\pi(a)))\end{aligned}$$

Q 8.1 Optimal policies in CQL

2. π is a probability distribution. What type of constraint does that imply about the sum of the elements of π? How could the objective then be written as a Lagrangian that includes this constraint?

Q 8.1 Optimal policies in CQL

2. π is a probability distribution. What type of constraint does that imply about the sum of the elements of π? How could the objective then be written as a Lagrangian that includes this constraint?

The sum of probabilities should be 1. In the Lagrangian we need to express the constraints as something =0, so we can take $\sum_a \pi(a) - 1 = 0$. The full Lagrangian with Lagrangian multiplier λ becomes $L(\pi, \lambda) = \sum_a \pi(a)(Q(a) - \log(\pi(a))) + \lambda\left(\sum_a \pi(a) - 1\right)$.

Q 8.1 Optimal policies in CQL

3. Take the derivative of the Lagrangian L with respect to $\pi(a_1)$. What does this imply about $\nabla_\pi L$?

Q 8.1 Optimal policies in CQL

3. Take the derivative of the Lagrangian L with respect to $\pi(a_1)$. What does this imply about $\nabla_\pi L$?

$$
\begin{aligned}
\nabla_{\pi(a_1)} L(\pi, \lambda) =& \nabla_{\pi(a_1)} \sum_a \pi(a)(Q(a) - \log(\pi(a))) + \lambda \left(\sum_a \pi(a) - 1 \right) \\
=& \nabla_{\pi(a_1)} \pi(a_1)(Q(a_1) - \log(\pi(a_1))) + \nabla_{\pi(a_1)} \lambda \pi(a_1) \\
& \text{(because all other terms in the sums are constant.)} \\
=& Q(a_1) - \log(\pi(a_1)) - 1 + \lambda \\
& \text{(the -1 is the result of the multiplication rule.)}
\end{aligned}
$$

Q 8.1 Optimal policies in CQL

4. Set the gradient to 0 and solve for π (you can again do this first for a single probability $\pi(a_1)$. Do you recognize the form of the solution?

Q 8.1 Optimal policies in CQL

4. Set the gradient to 0 and solve for π (you can again do this first for a single probability $\pi(a_1)$. Do you recognize the form of the solution?

$$
\begin{aligned}
Q(a_1) - \log(\pi(a_1)) - 1 + \lambda &= 0 \\
\log(\pi(a_1)) &= Q(a_1) - 1 + \lambda \\
\pi(a_1) &= \exp(Q(a_1)) \exp(-1 + \lambda) \\
&\propto \exp(Q(a_1))
\end{aligned}
$$

Since this for all actions we get a similar result, after normalization we see that

$$\pi(a_i) = \frac{\exp(Q(a_i)}{\sum_j \exp(Q(a_j))},$$

a softmax function.

Q 8.2 *Exam Question: Deep Q Networks

1. DQN uses a 'target network', a neural network with different parameters to the main 'Q network' to calculate the targets. What is the purpose of this network?

Q 8.2 *Exam Question: Deep Q Networks

1. DQN uses a 'target network', a neural network with different parameters to the main 'Q network' to calculate the targets. What is the purpose of this network?

   The purpose is to increase stability by reducing the feedback cycle/moving target that happens when the updated value of the q function also changes the targets the algorithm is trying to learn.

Q 8.2 *Exam Question: Deep Q Networks

2. Based on DQN's theoretical properties, would you expect the algorithm to have convergence guarantees? Briefly explain why (not), and how your answer relates to its good empirical performance.

Q 8.2 *Exam Question: Deep Q Networks

2. Based on DQN's theoretical properties, would you expect the algorithm to have convergence guarantees? Briefly explain why (not), and how your answer relates to its good empirical performance.

   Theoretically, DQN is an off-policy, semi-gradient bootstrapping, function approximation algorithm,and therefore liable to divergence due to the Deadly Triad. In practice, the authors of the DQN paper introduce some tricks to stabilise learning. These are:

   - Replay buffer: storing transitions and playing them back out of order to reduce correlations between subsequent transitions. This helps approximate the i.i.d. assumption of gradient descent,and with data-effficiency.
   - Target network: stabilising the target of the Q-learning update by using targets produced by a Q-network that has been frozen for a number of steps. This reduces variance in the weight updates compared to using a target that shifts as well with every weight update.
   - Error clipping: stabilise learning by not reacting wildly to large errors.

WELL DONE!

Tutorial 9 Overview

1. Off-policy RL with approximation exercises
2. Ask anything about HW4
   - Questions 8.2-8.3, 9.4

That's it!

See you next day